

Remember Glaze: A High Throughput Persistence Mechanism

Paul Louis Robert Quesnot

Abstract—Modern ledgers and databases still pair write-ahead logging with periodic snapshots, incurring long pauses, while current chunk-based approaches breaks down on bulk updates. Our proposition, *Remember Glaze*, removes both limits. It slices the state into fixed-size chunks stored in a circular ring log; each batch appends only the modified chunk plus its transaction list, giving constant-time commits and a strictly bounded disk footprint. Crash recovery touches at most one snapshot per chunk and replays only appropriate batches. On a 100 M-account benchmark this cuts median per-batch latency by 40–70 % against the previous standard and rebuilds the full state in a couple of minutes. *Remember Glaze* is therefore a practical drop-in for high-throughput systems that demand predictable latency and rapid restart.

I. INTRODUCTION

A. Motivation

Modern high-throughput systems such as blockchains and large databases are relied on to apply millions of updates per second and still recover fully after a crash. The dominant technique, write-ahead logging plus periodic full snapshots, meets crash-consistency requirements, however it introduces “latency cliffs”: long pauses whenever a gigabyte-scale snapshot is flushed to disk. With state sizes already in the multi-terabyte range (e.g., Ethereum archive nodes) even small per-batch slowdowns accumulate into hours of lost throughput each day. Despite this, no existing method simultaneously offers :

- 1) *Constant-time* commits independent of state size,
- 2) Graceful handling of arbitrarily large, non-interdependent writes, and
- 3) A deterministically bounded on-disk footprint.

Remember Glaze is motivated by this gap. The goal of this report is to design and evaluate a persistence layer that preserves simplicity while offering disk durability with *zero snapshot spikes*, predictable resource usage, first-class support for both bulk state transformations as well as programs leveraging spatial locality. Achieving these properties simultaneously would remove one of the main performance bottlenecks in next-generation database and blockchain infrastructures.

B. Contributions

Its main contributions can be summarized in the following points:

- **Function-centric batch commits:** *Remember Glaze* ensures that the complexity of batch transactions is solely determined by the applied state transformation functions, rather than by persistence mechanics.

- **State-size-independent performance:** Execution speed remains constant regardless of the state’s size, guaranteeing predictable and efficient processing times.
- **Deterministically bounded on-disk storage:** Our architecture precisely defines disk space requirements in advance, effectively preventing storage-related issues that arise from continuously expanding log files.
- **Decoupled transaction execution and persistence:** A clear architectural distinction between the execution of transactions and state persistence simplifies the system, improving maintainability and scalability.
- **Highly tunable execution:** Our model enables us to tune several parameters to optimize execution depending on its use cases.

In this report, we synthesize practical applications of *Remember Glaze*, thus providing empirically supported insights into the selection of optimal parameters to effectively balance the performance, latency, and recovery objectives.

II. BACKGROUND

Persistence and recovery strategies in modern data-intensive applications primarily follow two major paradigms: the previously mentioned write-ahead logging combined with periodic snapshotting as well as log-structured or incremental checkpointing approaches. This section details these methodologies, discusses their advantages and limitations, and introduces the *Glaze* and *WAL-snapshot* baselines to provide context for our proposed *Remember Glaze* solution.

A. Write-Ahead Logging (WAL) with Snapshots

This paradigm underpins numerous transactional database systems, such as Cassandra [6]. WAL ensures durability by logging every transaction to persistent storage before applying changes in memory. Periodically, these systems generate a complete snapshot of the current in-memory state, persisting it to disk and subsequently truncating the log. Recovery then involves loading the latest snapshot and replaying logged transactions to reconstruct the most recent consistent state [1].

While WAL-snapshot methods offer robust crash consistency and support fine-grained locking and rollbacks [1], their significant drawback lies in periodic snapshots, which incur substantial I/O overhead proportional to the total state size, resulting in latency spikes or “latency cliffs”. These pauses become critical performance bottlenecks, particularly in high-throughput, large-scale systems managing terabytes of state [5].

B. Log-Structured and LSM-Tree Architectures

Log-structured storage systems mitigate random write overhead by sequentially appending updates to a log. The seminal *Log-Structured File System (LFS)* [2] pioneered this concept, enhancing write performance by reducing random disk seeks. Extending this concept, *Log-Structured Merge-Trees (LSM-Trees)* structure data into sorted levels and periodically compact these levels to remove outdated entries [3]. Systems like LevelDB and RocksDB exemplify the LSM paradigm, delivering high sustained write throughput.

However, these systems experience periodic background compaction stalls, where compaction operations consume significant CPU and I/O resources. As data scales, compaction overhead can degrade overall system performance [3].

C. Incremental Checkpointing and Chunk-Based Approaches

Incremental checkpointing schemes, such as Apache Flink's incremental checkpoints [4] and Oracle's incremental buffer-pool checkpoints [5], address the drawbacks of full snapshots and background compactions by only persisting modified state portions since the last checkpoint. Such methods considerably reduce I/O and pause durations but still occasionally require full snapshots to ensure comprehensive recoverability.

The original *Glaze* design further advances incremental checkpointing by logically dividing the entire state into equally sized chunks, each stored in a circular buffer on disk. After processing each batch at a given epoch, a piece of the state that we will call a chunk along with write-sets, created from applying the batch to the state, are persisted. This strategy ensures constant-size writes per batch, bounding storage overhead and smoothing out latency. During recovery, *Glaze* loads the most recent snapshot of each chunk and replays corresponding logs to reconstruct the state incrementally.

Note that when we say write-set, we actually mean the state of a given address after a transformation was applied to it. For example let us have the address [1] with the value 6 associated to it. We apply addition to this address with the value 2. Our write-set for that operation then becomes $[1] \rightarrow 8$.

Despite improvements, *Glaze* encounters performance issues when processing large writes, such as extensive memset-type operations. These writes trigger massive write sets as with one operation we are creating a large amount of write-sets, thereby exponentially growing log files, creating substantial overhead for each affected batch.

D. Summary of Baseline Trade-Offs

- **WAL-Snapshot:** Ensures strong consistency and durability [1], but periodic full snapshots cause latency spikes proportional to state size [5].
- **Log-Structured and LSM-Tree Architectures:**[3] Achieves high sustained write throughput through sequential log appends [2], but incurs pronounced compaction-related stalls at scale [3].
- **Incremental / Chunk-Based (Glaze):** Offers bounded storage overhead and constant-size writes per batch, yet

large non-interdependent writes cause considerable per-chunk rewrite overhead.

III. IMPLEMENTATION AND DESIGN

A. Design

a) *General Structure:* Our design of *Remember Glaze* emphasizes efficiency and robustness in state persistence. The overall application state, represented as an array, is primarily kept in main memory to leverage fast access speeds. For durability, the system employs a circular ring buffer stored on disk, termed the *checkpoint ring log*, comprising of snapshot slots for storing discrete state chunks and associated function slots, which record batches of functions as they get executed, at the same epoch as the state chunk was persisted, much the same way *Glaze* works, only with the incoming execution messages instead of write-sets.

The performance that this design has comes with a cost, it requires good spatial locality in the operations executed, due to how the reconstruction works. It is meanwhile possible to create a *hybrid system*, that executes elements requiring high spatial locality with *Glaze*'s operations, saving the aforementioned write-sets, and otherwise executing on *Remember Glaze*'s algorithm. We argue that our design is still relevant seeing as most applications are built with spatial locality in mind and will continue this report under this assumption by detailing *Remember Glaze*'s design in a fine-grained manner:

b) *Separation of Transaction Processing and State Commitment:* A core design decision involves separating function execution from state persistence. The execution engine focuses purely on in-memory operations, optimized for cache locality and parallel execution. Meanwhile, the durability layer asynchronously serializes only the modified state chunk and its corresponding transaction log to disk after each batch execution, significantly reducing the overhead of persistent storage operations.

c) *Commitment:* When a new batch b of operations arrives, *Remember Glaze* first applies each operation in memory to only the affected chunk c , where $c = b \bmod N$, N the total number of chunks in the ring. Once this is completed, the system commits the changes by writing exactly one new snapshot of chunk c back into the circular ring buffer on disk. That chunk is linked to specific slot in the transaction log area holding metadata and operation payloads for batch b , occupying a slot in the checkpoint log.

For the next batch, the ring pointer advances to $(b+1) \bmod N$, so when you circle around after N batches, the oldest chunk and log entries are overwritten, making each batch commit consist of one copy of a single chunk plus one write of the batch log, no matter how large the entire state grows, every commit always writes exactly the same amount of data. This guarantees bounded disk usage and a constant, predictable persistence overhead per batch, eliminating any need for full-state snapshots or large, variable-length writes during steady-state processing.

Figure 1 shows how each batch is applied to memory and then flushed to disk *only* at the single chunk index $c = b \bmod N$.

Execution

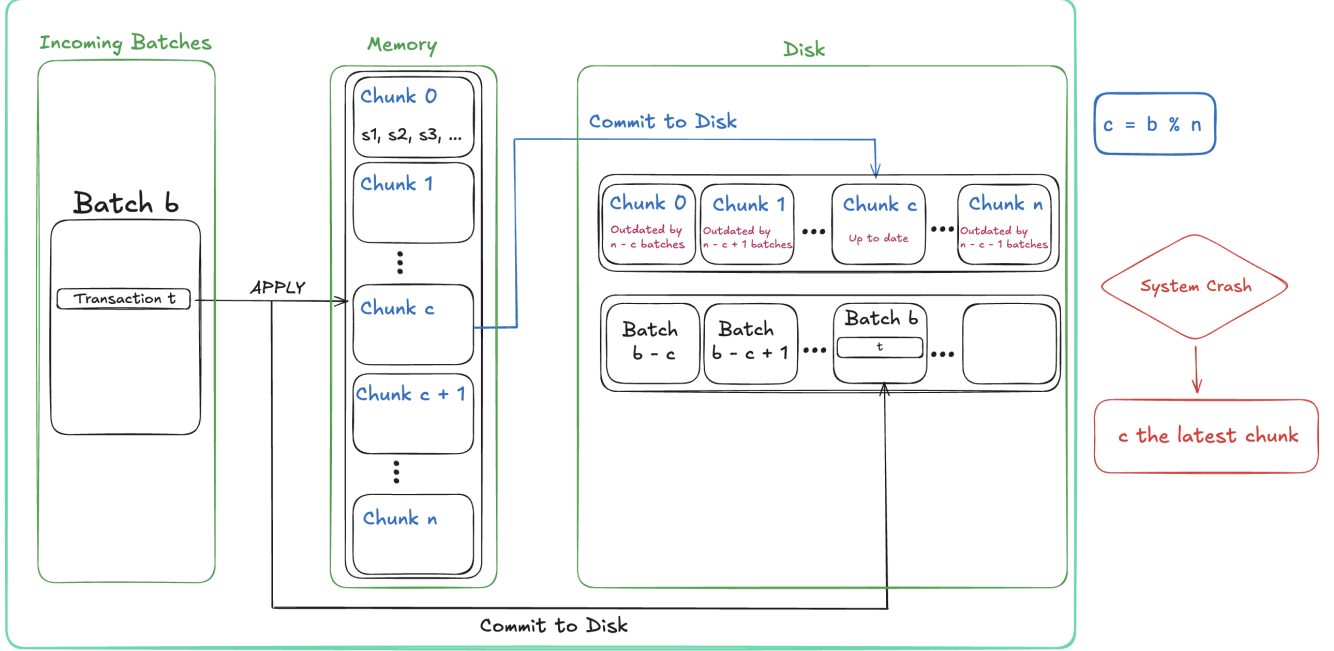


Fig. 1. Transaction processing in *Remember Glaze*. Incoming batches are applied in memory chunk by chunk; only the modified chunk is written into the circular ring buffer on disk. This guarantees that each batch’s persistence cost remains constant (independent of total state size), bounding I/O overhead.

As a result, while memory operations touch only chunk c , persistence happens via a fixed-size write into the circular buffer, eliminating full-state snapshots or large writes during steady-state execution.

d) Logical Time, Ring Geometry, and Epochs:

- **Epoch counter** (b). Every batch is assigned a monotonically-increasing epoch number $b \in \mathbb{N}$ by the scheduler. The commit protocol never rolls back an epoch once it has been *durably* recorded.
- **Ring index** ($c = b \bmod N$). A fixed modulus N maps epochs to one of N equally-sized *slots* in the checkpoint ring. Slot c simultaneously hosts:
 - 1) the *snapshot record* for state chunk c , and
 - 2) the *function record* for epoch b .

Because N is constant, the physical distance between two consecutive commits is also constant, giving truly state-size-independent I/O.

- **Overwrite discipline.** Epoch $b + N$ re-uses the same slot c once *every* chunk has advanced at least once. This “lazy garbage collection” ensures that no live snapshot is overwritten before a newer one has been persisted elsewhere.

e) *Recovery* : On startup, the *Remember Glaze* system restores its in-memory state by reading and replaying the checkpoint ring log. The recovery procedure proceeds in three main phases: (1) validate and parse the checkpoint header, (2) load the most recent chunks for each state region, and (3) replay any missing transactions to advance each chunk to the latest

epoch. By organizing chunks in a fixed-size circular buffer, the algorithm bounds both disk usage and recovery work. The pseudo-code for the algorithm is the following:

Algorithm 1 Recovery Algorithm (Pseudocode)

```

1: function RECOVERSTATEFROMLOG(log_fd)
2:   header  $\leftarrow$  READCHECKPOINTHEADER(log_fd)
3:   if header invalid or parameters mismatch then
4:     return Error: cannot recover
5:    $N \leftarrow \text{NUM\_STATE\_CHUNKS}$ 
6:   batchNum[0..N - 1]  $\leftarrow$  -1
7:   maxBatch  $\leftarrow$  -1
8:   latestIdx  $\leftarrow$  -1
9:   for  $k = 0$  to  $N - 1$  do
10:    snapSlot  $\leftarrow$  READCHUNKSLOT(log_fd, offset_for_chunk(k))
11:    if (snapSlot.chunk_idx ==  $k$ ) and
      (snapSlot.batch_num > batchNum[ $k$ ]) then
12:      batchNum[ $k$ ]  $\leftarrow$  snapSlot.batch_num
13:      if batchNum[ $k$ ] > maxBatch then
14:        maxBatch  $\leftarrow$  batchNum[ $k$ ]
15:        latestIdx  $\leftarrow$   $k$ 
16:   if latestIdx < 0 then
17:     return Error: no valid snapshot found
18:   Initialize stateArray  $\leftarrow$  0
19:   for  $k = 0$  to  $N - 1$  do
20:     if batchNum[ $k$ ]  $\geq$  0 then
21:       chunkData  $\leftarrow$  READSNAPSHOTDATA(log_fd,  $k$ )
22:       CopyIntoState(stateArray,  $k$ , chunkData)
23:   lastRecoveredBatch  $\leftarrow$  maxBatch
24:   for  $\Delta = 1$  to  $N - 1$  do
25:      $k \leftarrow (\text{latestIdx} - \Delta) \bmod N$ 
26:     for batch = batchNum[ $k$ ] + 1 to maxBatch do
27:       slotIdx  $\leftarrow$  batch mod  $N$ 
28:       txSlot  $\leftarrow$  READBATCHSLOT(log_fd, slotIdx)
29:       for each transaction  $\tau$  in txSlot.transactions do
30:         if  $\tau$  affects chunk  $k$  then
31:           APPLYTRANSACTION(stateArray,  $\tau$ )
32:   return stateArray, lastRecoveredBatch

```

Recovery

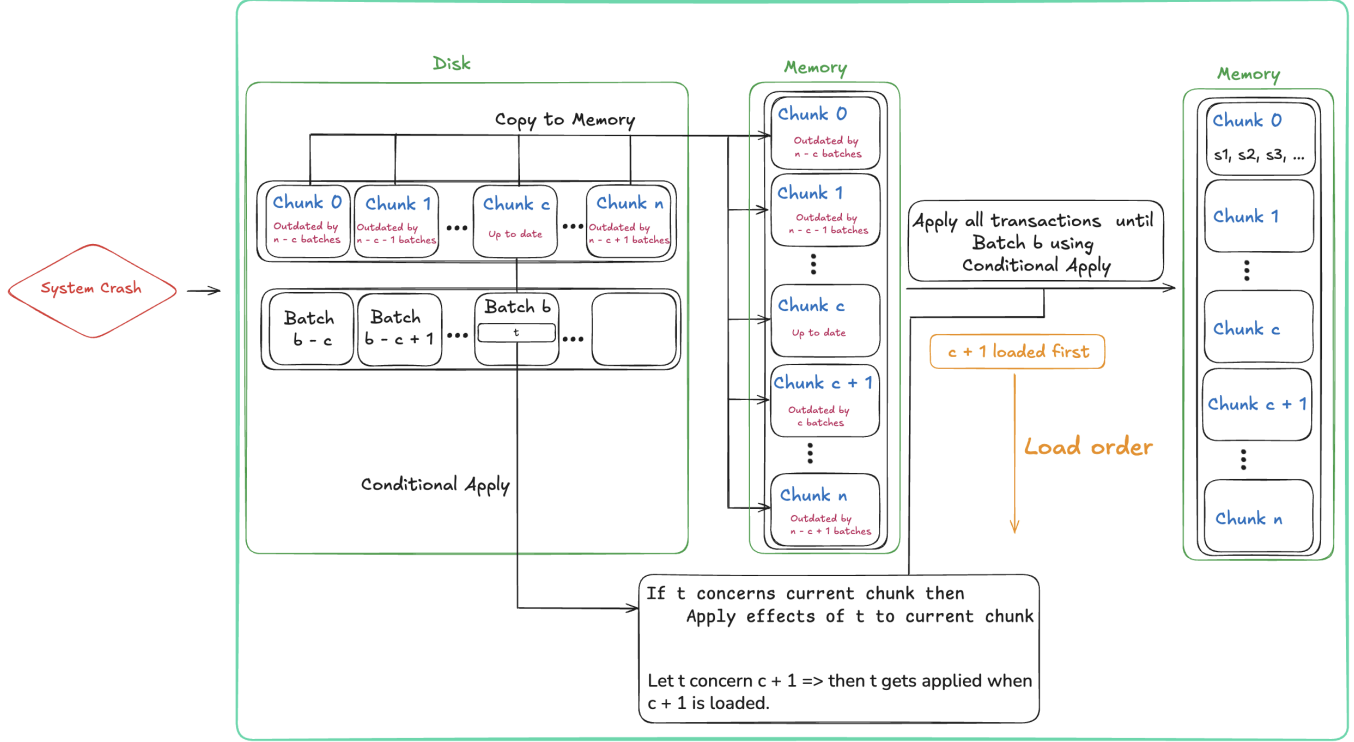


Fig. 2. Recovery in *Remember Glaze*. After a crash, all chunk slots on disk are first loaded into memory in reverse circular order (starting from the most recent chunk index c). Then, every transaction up to batch b in each chunk slot is conditionally applied to reconstruct the full state. Ultimately, only missing transactions (per chunk) are replayed, guaranteeing deterministic and bounded recovery work.

Figure 2 illustrates the recovery path and picks up from the system crash on figure 1. On startup, *Remember Glaze* scans the on-disk ring buffer to find the latest valid chunk index c (i.e., max batch number). It then copies each chunk's snapshot into memory—loading “chunk $c+1$ ” first (wrapping around), “chunk $c+2$ ”, and so on—so that each chunk ends up at its most recent snapshot. Finally, it conditionally replays any transactions whose batch numbers exceed that chunk's snapshot batch, guaranteeing that the in-memory state is fully up to date with batch b without ever doing a full-state read or log replay. The mentioned conditional replay is key here and means that our current operation is applied if and only if it has an effect on the chunk we're currently restoring.

f) Complexity Analysis: This design ensures that recovery work is bounded by $O(NS + N^2B)$, where N is the number of chunks (for loading snapshots), S is the size of each account, and NS is therefore the total size of the chunks in bytes in the chunkSlots. B is the batch size in number of transactions. Phase 2 reads and copies one chunk snapshot for each of the N chunks, so the cost is $O(NS)$. Phase 3 replays at most N batches for each chunk; each batch contains at most B transactions, giving $O(NNB) = O(N^2B)$. Header parsing and the initial metadata scan add only $O(N)$ and do not change the leading term. Hence the worst-case recovery work is $O(NS + N^2B)$. In typical random-access workloads the replay term drops to $O(NB)$

because only roughly $1/N$ of the transactions affect a given chunk.

By never writing a full state dump and instead relying on fixed-size chunk slots plus transaction slots, *Remember Glaze* guarantees a predictable on-disk footprint and a deterministic recovery path.

Critical design parameters, such as chunk size, batch size, and the number of state chunks, are configurable at compile-time. These parameters represent important trade-offs between steady-state processing speed and recovery complexity, allowing the system to be tailored to specific operational requirements.

B. Implementation

The implementation of *Remember Glaze* is designed for simplicity and ease of reproducibility and was done in the C language. The codebase is structured into clear, logical sections:

1) Structs:

a) CheckpointHeader: This structure defines the metadata stored at the beginning of the checkpoint log file. It includes fields for verifying the integrity and compatibility of the checkpoint (through a magic number), specifying the log version, and providing constants like the total number of state chunks and the accounts per chunk, crucial for correct state reconstruction during recovery.

```
typedef struct {
```

```
uint32_t magic;
uint32_t version;
uint32_t current_cycle_ptr;
uint32_t num_state_chunks;
uint32_t accounts_per_chunk;
} CheckpointHeader;
```

b) *Transaction*: The Transaction structure represents state updates compactly. It encodes sender and receiver information, including their corresponding operations (transfers or bulk updates), using packed integers. The amount field carries either the transaction value or the payload for bulk update operations.

```
typedef struct {
uint64_t sender;
uint64_t receiver;
uint64_t amount;
} Transaction;
```

c) *ChunkSlot*: The ChunkSlot represents a fixed-size snapshot stored in the on-disk ring buffer. Each chunk corresponds to a segment of the state, identified by `chunk_idx_in_ring`, and is associated with a specific batch number, facilitating precise state recovery by loading the correct snapshot and replaying subsequent transactions.

```
typedef struct ChunkSlot {
uint32_t batch_num;
uint32_t chunk_idx_in_ring;
int64_t state_data[ACCOUNTS_PER_STATE_CHUNK];
} ChunkSlot;
```

d) *BatchSlot*: BatchSlot encapsulates a collection of transactions processed in a single batch. Each slot includes the batch number and the actual transaction records. It serves as the primary log entry unit used during the replay phase in recovery, ensuring accurate and ordered state reconstruction.

```
typedef struct BatchSlot {
uint32_t batch_num;
uint32_t tx_count;
Transaction transactions[BATCH_SIZE];
} BatchSlot;
```

e) *StateHash*: StateHash stores a lightweight integrity check of the current state. This hash value, computed using a parity-based scheme, provides a quick validation mechanism to ensure the correctness of the state following recovery or incremental updates, complemented by the batch number it references.

```
typedef struct {
uint64_t hash;
uint32_t batch_num_for_hash;
} StateHash;
```

f) *CommitThreadArgs*: This structure bundles arguments passed to the asynchronous commit worker thread. It includes details such as the slot index in the checkpoint ring, the current batch number, pointers to memory-mapped regions for efficient disk writes, transaction counts, and copies of the transaction and state data. This separation allows the main thread to proceed with transaction processing without delays from persistence operations.

```
typedef struct {
uint32_t slot_idx_in_cycle;
uint32_t current_batch_num;
void *mapped_log_region;
long pagesize;
uint32_t num_tx_in_batch;
int64_t state_chunk_data
[ACCOUNTS_PER_STATE_CHUNK];
Transaction transactions_copy[BATCH_SIZE];
} CommitThreadArgs;
```

2) *Utility functions*: Utility functions include optimized memory-copy operations to exploit hardware capabilities, hash computations for state verification, and diagnostic routines for debugging. The persistence layer manages all interactions with disk storage, including initializing and managing the checkpoint ring log, writing snapshots, and applying transactions. Each persistence operation employs POSIX calls, such as `mmap` and `msync`, to minimize I/O overhead and ensure atomicity.

3) *Transaction Processing*: Concurrency in *Remember Glaze* is implemented using POSIX threads (pthread library). The transaction batch is processed by a single workout for consistency and to allow nonassociative operations. The processing itself consists of calling an "apply" function, which is a black box, decoding the transactions as executable functions which are then appropriately applied to the said state.

4) *Synchronization*: Synchronization points occur minimally, mainly at batch boundaries before state persistence, and are handled by a worker thread which commits the combination of chunk and log files to the disk. The file system structure is straightforward, comprising the checkpoint log file for durable state storage, transaction logs, optional checksum files, and diagnostic outputs. By leveraging as much as possible the capabilities of local storage, the implementation hopes to achieve low-latency persistence and recovery.

Portability and ease of use were also prioritized. The implementation relies exclusively on standard POSIX interfaces, and compile-time flags facilitate parameter adjustments and performance optimizations tailored to different deployment scenarios.

5) *Recovery*: We will now detail the inner workings of the recovery algorithm that we implemented:

Header Validation: Read a fixed-size header at file offset 0, verify a magic constant and expected parameters (number of chunks, accounts per chunk). Abort if validation fails.

```
int recover_state_from_log(int log_fd,
                          int64_t *restrict state,
                          int *last_batch)
{
    CheckpointHeader h;
    if (pread(log_fd, &h, sizeof h, 0) != sizeof h ||
        h.magic != CHECKPOINT_MAGIC ||
        h.num_state_chunks != NUM_STATE_CHUNKS ||
        h.accounts_per_chunk != ACCOUNTS_PER_STATE_CHUNK
    )
        return -1;
```

Find Latest Snapshot: For each chunk index $k \in [0..N-1]$, we read its ChunkSlot record at the offset `OFFSET_CHUNK_SLOT(k)`. If `ChunkSlot.chunk_idx = k`, we record `batchNum[k] = ChunkSlot.batch_num`. We track the maximum `batch_num` across all valid slots, and let `latestIdx` be the chunk index that holds that maximum batch.

```
int latest_chunk = -1, latest_batch = -1;
for (uint32_t k = 0; k < NUM_STATE_CHUNKS; ++k) {
    ChunkSlot slot;
    off_t off = CHECKPOINT_HEADER_SIZE + k * sizeof
    slot;
    if (pread(log_fd, &slot, sizeof slot, off) ==
    sizeof slot &&
        slot.chunk_idx_in_ring == k &&
        (int)slot.batch_num > latest_batch) {
        latest_batch = slot.batch_num;
        latest_chunk = k;
    }
}
if (latest_chunk < 0) return -1;
```

Load Chunked State: We zero out the in-memory `stateArray`. Then, for each k with `batchNum[k] ≥ 0`, copy its `ChunkSlot.state_data` (fixed-size array of `ACCOUNTS_PER_CHUNK` elements) into the corresponding region of `stateArray`. This reconstructs the partial snapshot up to each chunk’s own batch.

```
memset(state, 0, PADDED_ACCOUNT_COUNT * ACCOUNT_SIZE
);
int batch_of_chunk[NUM_STATE_CHUNKS];
for (uint32_t k = 0; k < NUM_STATE_CHUNKS; ++k) {
    ChunkSlot slot;
    off_t off = CHECKPOINT_HEADER_SIZE + k * sizeof
    slot;
    if (pread(log_fd, &slot, sizeof slot, off) ==
    sizeof slot &&
        slot.chunk_idx_in_ring == k) {
        nt_memcpy(state + k *
        ACCOUNTS_PER_STATE_CHUNK,
            slot.state_data,
            ACCOUNTS_PER_STATE_CHUNK *
        ACCOUNT_SIZE);
        batch_of_chunk[k] = slot.batch_num;
    } else {
        batch_of_chunk[k] = -1;
    }
}
*last_batch = latest_batch;
```

Replay Transactions: We let `maxBatch = maxk batchNum[k]`, which is the start from chunk index $(\text{latestIdx} - 1) \bmod N$ and move backward (wrapping around), for each chunk k , replaying all batches $(\text{batchNum}[k] + 1, \dots, \text{maxBatch})$. Each batch is stored in a `BatchSlot` at offset `OFFSET_BATCH_SLOT(i)` where i is the batch number. Within each `BatchSlot`, we iterate over each transaction up to `tx_count` transactions; if a transaction touches an account index belonging to chunk k , we apply its update (subtract from sender, add to receiver, or perform

a range set) directly into `stateArray`. We continue until all chunks have been brought uniformly to `maxBatch`.

```
size_t base = CHECKPOINT_HEADER_SIZE +
    NUM_STATE_CHUNKS * sizeof(ChunkSlot);
uint32_t cur = (latest_chunk + NUM_STATE_CHUNKS - 1)
    % NUM_STATE_CHUNKS;
for (
    uint32_t done = 0;
    done < NUM_STATE_CHUNKS - 1;
    ++done)
{
    cur = (cur + NUM_STATE_CHUNKS - 1) % NUM_STATE_CHUNKS
    for (
        uint32_t b = batch_of_chunk[cur] + 1;
        b <= latest_batch; ++b)
    {
        off_t off = base +
            (b % NUM_STATE_CHUNKS) * sizeof(BatchSlot);
        BatchSlot txslot;
        if (pread(
            log_fd,
            &txslot,
            sizeof txslot,
            off) != sizeof txslot)
            continue;

        for (
            uint32_t t = 0;
            t < txslot.tx_count;
            ++t)
        {
            const Transaction *tx =
                &txslot.transactions[t];
            /* apply transfer or range-set if this chunk owns the
               addresses */
        }
    }
    return 0; /* success */
}
```

The implementation is easily compiled and executed, producing detailed performance metrics suitable for the evaluation described in the subsequent sections of this report.

IV. EVALUATION

A. Setup

1) *Hardware:* The hardware used for testing our system was the *c7gd.8xlarge* on AWS, running on ARM. Its specs are listed in the following table:

TABLE I
AWS C7GD.8XLARGE INSTANCE SPECIFICATIONS

Specification	Details
Instance Type	c7gd.8xlarge
vCPUs	32
Memory	64 GiB
CPU Architecture	Arm64 (AWS Graviton3)
Base Clock Speed	2.6 GHz
Hypervisor	AWS Nitro
Local Storage	1.9 TB NVMe SSD
Network Bandwidth	Up to 15 Gbps
EBS Bandwidth	Up to 10 Gbps
Total Storage Volume	1908 GiB

This system was used for the following reasons:

- **Abundant System Memory (64 GiB):** The 64 GiB of RAM allows for all of the system’s state to reside in memory, drastically reducing latency and accelerating operations that require frequent access to account data. This is particularly beneficial for the in-memory processing aspects of our algorithms.
- **Local NVMe SSD Storage (1.9 TB):** Because both baseline schemes and Remember Glaze generate heavy write traffic, continually flushing memory images and appending multi-megabyte logs, the node relies on a 1.9 TB local NVMe SSD, which sustains 1.3 GiB / s of throughput while retaining low access latency under mixed random and sequential workloads, critical for these experiments.
- **Representative Cloud Environment:** Utilizing a high-performance instance like the *c7gd.8xlarge* reflects a typical deployment environment for modern, scalable data management systems in the cloud. This ensures that our experimental findings are relevant and applicable to real-world scenarios where such systems would be deployed.

2) *Software:* The hardware described above ran an instance of Ubuntu 24.04 LTS (HVM). All executed code was written in C, compiled using *gcc 13.3.0* with the *-O3* compiler optimization flag, enabling aggressive performance optimizations suitable for test builds. These tests primarily used a standard chunk size of 512 KB unless otherwise stated, with each entry in the state being 8B in size. Each batch contained 2^{16} transactions, each transaction sized at 24B, making each batch approximately 1.5MB. Tests varied in the number of accounts and batches to explore different operational scenarios and highlight key performance trade-offs.

The implementation leveraged the following C libraries for various system functionalities:

- Standard Input/Output: `stdio.h`
- Standard Library Functions: `stdlib.h`
- String Operations: `string.h`
- Integer Types: `stdint.h`, `inttypes.h`
- Mathematical Functions: `math.h`
- System Calls and File Operations: `unistd.h`, `fcntl.h`, `sys/types.h`, `sys/stat.h`
- Error Handling: `errno.h`
- Memory Management: `sys/mman.h`
- Threading and Concurrency: `pthread.h`, `omp.h`
- Time Management: `time.h`
- Boolean Data Types: `stdbool.h`

We also manually defined the source for the *POSIX* our implementation used, which was 200809L, corresponding to *POSIX.1-2008 with the 2008 Technical Corrigendum 1*. This was used for portability reasons as well as making dependencies explicit.

The libraries listed above collectively provided functionality for file I/O, memory management, concurrency control, time tracking, and system-level operations necessary for high-

performance state management and persistence mechanisms evaluated in this report.

a) *Operation generation:* To thoroughly evaluate the performance of *Remember Glaze* and compare it against the baseline algorithms, a custom operation generator was developed. This tool allowed for precise control over the characteristics and scale of the synthetic workloads used in our experiments, such as percentage of operations in each batch that was a *memset* operation, i.e a large non-interdependent write as well as the size of that write compared to our ”standard” operations which simulated a transaction between two parties, where one person’s ”account” gets ”debited” and the other ”credited” atomically.

The transaction generation process leverages the OpenMP library for parallel execution across multiple CPU cores, significantly speeding up the creation of large transaction files. Each thread uses `rand_r` with a unique seed derived from the system time and its OpenMP thread ID, ensuring thread-safe and statistically diverse random number generation within a single run. While the use of time introduces variability in the exact sequence of random numbers across different runs, the overall statistical properties of the generated workload (e.g., distribution of operation types, account indices, and amounts) remain consistent, allowing for performance analysis. For the sake of consistency, the generated `transactions.bin` file is used across comparable test runs.

We will now examine the performance of our model, *Remember Glaze*, comparing it to the current state of the art listed in the Background section. This direct comparison highlights *Remember Glaze*’s advantages in handling specific challenges related to persistent state updates and recovery.

B. Experiment 1: How does Remember Glaze compare to Glaze in handling bulk writes?

1) *Introduction:* As mentioned previously, a chief limitation of the original *Glaze* design is its poor behavior when a batch contains wide, non-interdependent updates, for example a contract that overwrites an entire range of accounts with a single *memset*-style operation. Every such update forces *Glaze* to append a disproportionately large log entry, ballooning I/O and stalling the pipeline. Experiment 1 quantifies the extent of that bottleneck and tests the claim that *Remember Glaze* removes it.

a) *Hypothesis:* Because *Remember Glaze* commits *functions*, not raw byte deltas, the cost of a bulk write should be *constant*, independent of the span of memory affected. We therefore expect its runtime to grow only moderately with the share of *memset* transactions, whereas the baseline *Glaze* should degrade quasi-exponentially.

b) Methodology:

- **State size:** 5 000 000 accounts split into 512 KB chunks ($N = 10\,240$).
- **Workload:** 5 000 batches, each holding 2^{16} transactions. We sweep the fraction of bulk-write operations from 0% to 100% in powers of two (0, 1, 2, 4, . . . , 100%).

- **Bulk-write shape:** every memset sets a 64 KB contiguous slice chosen at random inside the chunk to a random value, mimicking a large contract initialization or account prune. A 64 KB range is (i) large enough to trigger Glaze’s exponential growth pathology and (ii) small enough to keep a single end-to-end run under 45 min on our AWS budget.

c) *Metrics captured.*: We measure the *clock time* to process all 5000 batches, averaged over five independent runs. Figure 3 plots this total runtime against the percentage of memset transactions, providing a direct visual of how each persistence scheme scales with bulk-write intensity.

2) *Results:* The comparative runtime results of *Remember Glaze* and *Glaze* under increasing proportions of bulk writes are depicted in Figure 3.

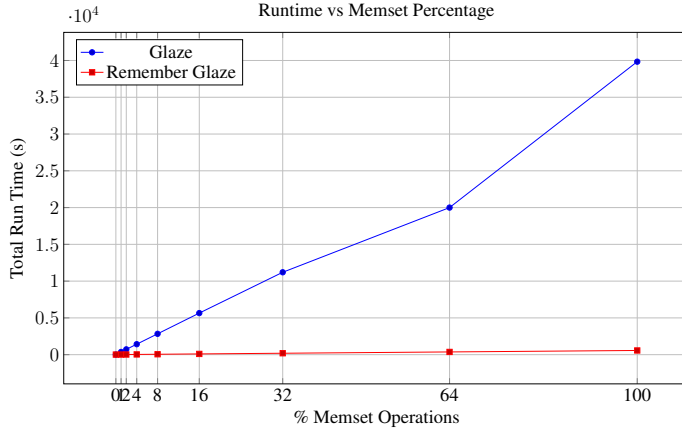


Fig. 3. Runtime comparison for Glaze (red) and Remember glaze (blue) as the percentage of memset function calls increases.

3) *Observations:* From Figure 3, a conspicuous divergence between *Glaze* and *Remember Glaze* emerges as the proportion of bulk write transactions increases. Initially, at 0% bulk writes, both systems show comparable performance. However, even a small increment in bulk writes drastically impacts *Glaze*, as observed at merely 1% bulk operations, execution time jumps from approximately 15 seconds to over 360 seconds. This exponential growth pattern continues as the bulk write percentage rises, reaching over 11,000 seconds at 32%, and nearly 40,000 seconds at 100% bulk operations.

In stark contrast, *Remember Glaze* demonstrates resilience to increasing percentages of bulk write. Its runtime remains comparatively low and scales linearly rather than exponentially. Even at 100% bulk writes, *Remember Glaze*’s total execution time remains approximately 570 seconds—around 70 times faster than *Glaze* under identical conditions. This demonstrates a clear performance advantage in handling large non-interdependent writes.

4) *Deductions:* The results conclusively validate the primary motivation behind *Remember Glaze*. The exponential performance degradation observed in *Glaze* highlights its inherent inefficiency in managing bulk writes. This inefficiency arises from *Glaze*’s design, which mandates reading and rewriting

entire chunks for each bulk write, severely amplifying I/O overhead and latency as bulk operations increase.

In contrast, *Remember Glaze* employs a function-centric commit mechanism, treating bulk writes as atomic, constant-time operations, with the only added complexity coming from the operation itself. This design effectively eliminates the compounding I/O overhead characteristic of the original *Glaze*, confirming that handling wide, non-interdependent writes as first-class operations is essential for maintaining high performance and stable latency.

Nevertheless, an underlying constraint remains pertinent, write functions that depend on another address’ value, what we call interdependent functions, must exhibit spatial locality within individual chunks to fully leverage the benefits of *Remember Glaze*’s design. Such operations that span multiple chunks or lack spatial locality wouldn’t be recovered correctly due to the semantics of the algorithm discussed earlier. Therefore, careful tuning of chunk sizes and strategic spatial allocation of interdependent functions are critical considerations in practical deployments.

C. *Experiment 2: How do the two approaches behave as the application state grows from a few million to one hundred million accounts?*

1) *Introduction:* While Experiment 1 isolated the benefits of *Remember Glaze* for bulk updates, real deployments must also decide whether it is worth replacing the well-understood *WAL-snapshot* paradigm that still dominates mainstream databases.

a) *Hypothesis.*: Because every *WAL-snapshot* commit eventually triggers a state-proportional checkpoint, we expect its throughput and latency to degrade almost linearly with state size. By contrast, seeing as *Remember Glaze* writes one fixed-size chunk per batch, its steady-state cost should stay almost constant; the price paid will surface in crash recovery, which must replay more individual chunks as the state expands.

b) *Methodology.*: Both engines are fed an identical synthetic workload of 50 000 batches (2^{16} transactions per batch, 95 % transfers and 5 % 64 KB range updates). Key parameters are held constant to ensure a fair comparison:

- **Chunk size** in *Remember Glaze*: 512 KB (shown in Experiment 3 to sit near the throughput sweet spot).
- **Snapshot interval** in the *WAL-snapshot* setup: every 1 000 batches, matching common production practice.

State size is scaled across six points— 2, 5, 10, 20, 50, and 100×10^6 accounts—to expose any non-linearities.

c) *Metrics captured.*:

- 1) *Total execution time* for all 50 000 batches, reflecting aggregate throughput (Fig. 4).
- 2) *Median per-batch latency*, highlighting steady-state pause behaviour (Fig. 5).
- 3) *Full-crash recovery time*, measured from process start to the first successful post-recovery commit (Fig. 6).

Repeating each configuration five times and reporting the mean eliminates noise from the host hypervisor and SSD wear-leveling. The resulting data provide a head-to-head picture

of the scalability limits and operational trade-offs of both persistence schemes.

2) *Results*: The experimental results are presented in Figures 4, 5 and 6 showing the total execution time, per-batch runtime and recovery runtime as a function of the number of account.

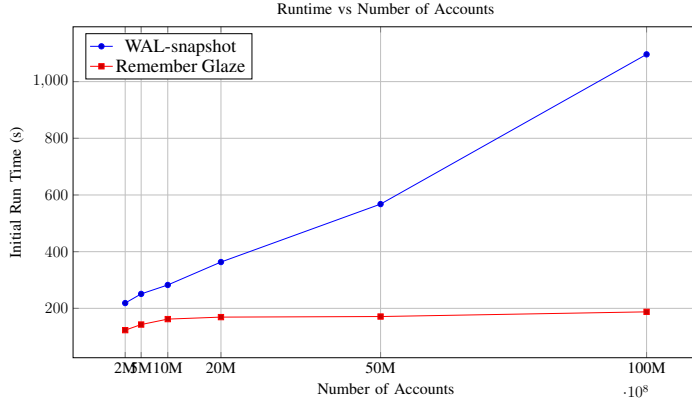


Fig. 4. WAL-snapshot's runtime in blue, *Remember Glaze*'s in red as the size of the state increases.

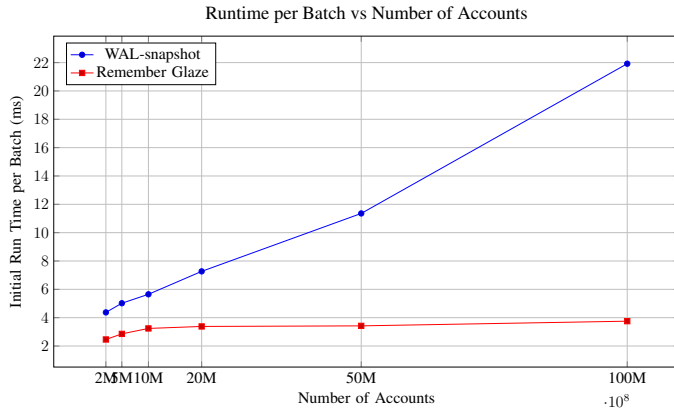


Fig. 5. WAL-snapshot and *Remember Glaze*'s runtimes as the number of elements in our state increases.

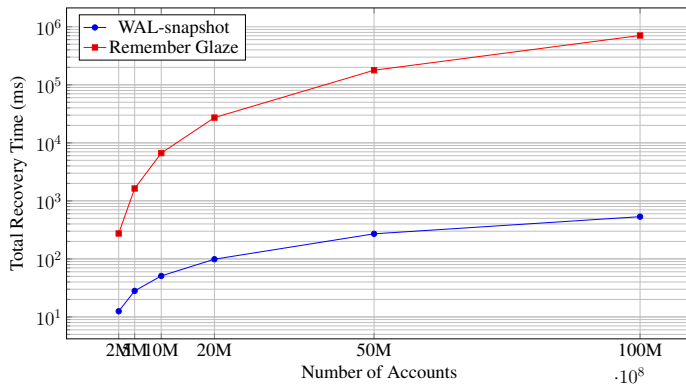


Fig. 6. Comparing *Remember glaze*'s and WAL-snapshot's recovery algorithms

3) *Observations*: The performance metrics clearly illustrate significant differences between the two approaches. As state

size grows, the total execution time of *WAL-snapshot* increases substantially faster than *Remember Glaze*. At 100 million accounts, *WAL-snapshot* takes over five times longer than at 2 million accounts, highlighting its linear scalability constraint and considerable overhead due to snapshotting operations. In contrast, *Remember Glaze*'s total execution time demonstrates only moderate growth, maintaining relatively stable per-batch latencies, irrespective of state size.

Further analysis of per-batch execution times confirms that *Remember Glaze* maintains consistently low latencies, showing only minor increases as the number of accounts expands significantly. Conversely, *WAL-snapshot* shows an increasingly steep rise in per-batch latency, indicating a marked performance degradation under large-scale workloads.

The third graph further illuminates the performance characteristics, specifically highlighting the recovery phase. *WAL-snapshot* demonstrates a markedly superior recovery performance with recovery times growing relatively modestly, even at substantial account counts. In stark contrast, *Remember Glaze* experiences exponential growth in recovery time as the number of accounts increases. At 100 million accounts, the recovery time for *Remember Glaze* is dramatically higher compared to *WAL-snapshot*, underscoring a key limitation of the architecture.

4) *Deductions*: These results validate the architectural advantages of *Remember Glaze*. Specifically, its ability to perform constant-time, fixed-size writes per batch, irrespective of total state size, substantially reduces latency spikes inherent in *WAL-snapshot* architectures.

The *WAL-snapshot* method's need for periodic full state snapshots imposes a weighty performance penalty, especially notable in large-scale environments. This limitation can cause detrimental effects on high-throughput, latency-sensitive workloads, emphasizing the suitability of *Remember Glaze* for modern applications requiring predictable, low-latency persistence.

However, a trade-off remains evident in the recovery phase. While *Remember Glaze* achieves superior runtime performance under steady-state conditions, its recovery complexity grows rapidly with increasing state size due to the need to reconstruct the state from multiple small chunks. The third graph explicitly demonstrates this exponential growth in recovery time, highlighting the critical importance of optimizing recovery mechanisms. Consequently, tuning parameters such as chunk size become essential to balancing regular performance against recovery efficiency. Additionally, exploring approaches like rolling super-chunks or background checksum computations could considerably alleviate the identified bottleneck in recovery performance.

D. Experiment 3: What effects do the chunk sizes have Remember Glaze's performance in recovery and transaction processing?

1) *Introduction*: Chunk size is the single most powerful "dial" exposed by *Remember Glaze*: it determines (i) how much data must be written to the checkpoint ring on every commit and (ii) how many distinct snapshots and log fragments have

to be scanned during crash recovery. Experiment 3 therefore investigates *how varying the chunk size influences both steady-state throughput and recovery latency*.

a) *Hypothesis.*: Increasing the chunk size should shorten recovery—because fewer, larger chunks have to be loaded and replayed, however this comes at the cost of higher per-batch I/O, which will in turn inflate processing time and median batch latency. We expect to observe a clear trade-off curve with a “sweet spot” that balances the two opposing effects.

b) *Methodology.*: We fix all other parameters and vary the chunk size across seven powers of two (256;512;1024;2048;4096;8192;16384) KB. For each chunk size we run an identical synthetic workload of 50,000 batches, each batch containing 2^{16} transactions (95 % random pairwise transfers, 5 % memset-style range updates). The experiment is repeated for four state sizes—1, 10, 50, and 100×10^6 accounts—to assess how the trade-off shifts with scale. Every configuration is executed five times on the same c7gd.8xlarge instance; we report the mean values.

c) *Metrics recorded.*:

- **Recovery time for each run** (Fig. 7): time to restore the full state and reach the first post-crash commit.
- **Total processing time** over all 50,000 batches (Fig. 8).
- **Median per-batch latency** measured at the 50th percentile (Fig. 9), which filters out occasional GC or OS-level noise and highlights the systematic cost of larger snapshots.

This controlled sweep isolates the impact of chunk size and provides concrete guidance for operators looking to tune *Remember Glaze* to a given throughput target or recovery-time objective.

2) *Results.*: The results for this experiment are in figures 7 through 8 and show the effects of chunk sizes on state recovery speed as well as general processing performance.

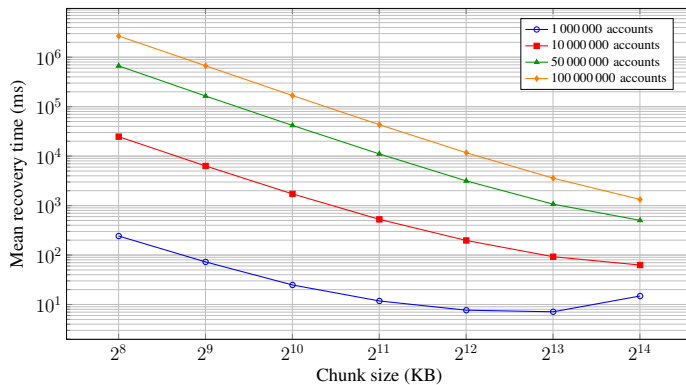


Fig. 7. Recovery time as the size of the chunks increases.

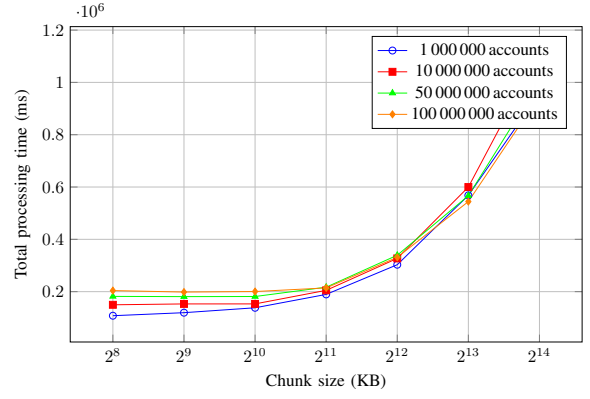


Fig. 8. Processing time as the size of the chunks increases.

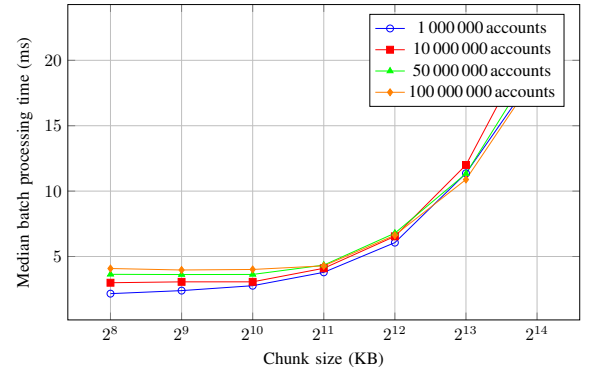


Fig. 9. Median per-batch latency versus chunk size

3) *Observations.*: Figures 7–9 reveal a clear, antagonistic relationship between chunk size and the two main performance axes of *Remember Glaze*:

- 1) **Recovery performance improves super-linearly with larger chunks.** For every workload size the mean recovery time drops by almost two orders of magnitude when moving from 256 KB to 4 MB chunks, and continues to fall—albeit more gently—up to 8. Beyond that point the curve flattens (and even rebounds slightly for the smallest dataset). This is because of the fact that at that point 16 MB is greater than the size of the state.
- 2) **Steady-state throughput deteriorates roughly linearly with chunk size.** The total runtime in Fig. 8 stays almost flat up to 1 MB, then rises sharply; at 16 MB chunks the end-to-end job completes $\approx 7\text{--}9\times$ slower than with 256 KB chunks. The same trend appears in the median per-batch latency (Fig. 9), which climbs from $\sim 2\text{--}4$ ms at sub-MB chunks to $\sim 18\text{--}22$ ms at 16 MB.
- 3) **The “knee” of the curve shifts with state size.** Larger datasets profit longer from increasing the chunk size: for 100×10^6 accounts the recovery curve keeps falling up to 16 MB, whereas for 1×10^6 accounts the optimal point is already reached around 8 MB. In contrast, batch latency begins to climb after 1 MB almost independently of dataset size.

4) *Deductions*: The data confirm the intuitive trade-off built into the design:

- *Why recovery accelerates with larger chunks*. Fewer, bigger chunks translate to fewer replay passes during crash recovery. Because the recovery algorithm is $O(N_{\text{chunks}})$ for snapshot loading and $O(N_{\text{chunks}}^2 \times B)$ for log replay, halving N_{chunks} roughly quarters the amount of work in both phases, explaining the near-hyperbolic drop in Fig. 7.
- *Why processing slows down with larger chunks*. Every commit writes an entire chunk snapshot. As the chunk grows, the amount of data copied from DRAM to the NVMe device (and checksummed) scales linearly; once the snapshot no longer fits in the LLC, memory bandwidth—not SSD throughput—becomes the bottleneck, producing the quasi-exponential wall in Figs. 8 and 9.
- *Choosing an operating point*. For small and medium deployments (up to $\sim 10^7$ accounts) a chunk size between 512 KB and 1 MB offers a sweet spot: recovery remains below one second while batch latency stays within the 3 ms envelope needed for high-throughput workloads. At very large scales the pendulum swings: moving to 4 MB–8 MB chunks reduces worst-case recovery from minutes to seconds at the cost of single-digit-millisecond extra latency per batch—often an acceptable trade-off for chains or ledgers that prioritise fast restart times.
- *Implications for tuning*. Because the optimal chunk size depends on both state size and recovery-time objective (RTO), a static compile-time constant is sub-optimal. A promising avenue is **adaptive chunking**: the system could start with small chunks for low latency, then merge them into “super-chunks” in the background once the state crosses a configurable threshold or during periods of low traffic, thereby shortening future recoveries without impacting peak throughput.

In summary, Experiment 3 quantifies the fundamental dial exposed by *Remember Glaze*: tuning chunk size lets operators trade a linear increase in steady-state latency for an inverse-exponential decrease in crash recovery time. Practical deployments should therefore pick the smallest chunk that still meets their RTO, or implement an adaptive policy that converges on that value automatically.

E. Conclusion

This evaluation demonstrates that *Remember Glaze* effectively achieves its primary objectives: maintaining consistently high throughput and low latency for transaction processing, particularly under workloads involving frequent, large-scale bulk updates such as memset-type operations. Compared to the conventional WAL-snapshot architecture, *Remember Glaze* significantly improves batch-processing performance, achieving a 40–70% reduction in batch-execution times across all tested state sizes. Furthermore, it entirely eliminates the periodic latency spikes typically caused by the snapshotting of large data sets, thus ensuring smoother and more predictable performance.

Additionally, when benchmarked against the original *glaze* algorithm, *Remember Glaze* consistently sustains stable op-

eration even under intense workloads that previously led to performance degradation. This highlights the effectiveness of the design choices made to specifically accommodate bulk-update operations.

Nevertheless, these performance enhancements introduce certain trade-offs. The chunk-based architecture, while advantageous during routine batch processing, results in increased complexity and time required for state recovery after crashes or system restarts. Specifically, the recovery duration scales proportionally with the number of chunks, which is directly influenced by both the overall state size and the chosen chunk dimensions. Consequently, recovery performance can degrade markedly for very large deployments involving tens or hundreds of millions of accounts, potentially exceeding the recovery times associated with the WAL-snapshot method.

An important implication is that the choice of chunk size becomes critical. Smaller chunks reduce the overhead of real-time persistence and improve regular transaction throughput, whereas larger chunks substantially speed up recovery procedures by minimizing the total chunk count. Selecting an optimal chunk size is therefore workload-specific and might require dynamic adjustments during different phases of system deployment.

In conclusion, *Remember Glaze* represents a practical and effective compromise between standard snapshot-based systems and fully in-memory solutions. It is particularly suitable for applications demanding high performance in transaction processing with frequent bulk updates, such as modern blockchain systems or real-time analytics platforms. While recovery complexity presents a notable challenge, it is manageable through thoughtful system tuning and design trade-offs. Future improvements might explore adaptive chunk sizing, hybrid recovery mechanisms to improve recovery times, and enhanced support for functions spanning multiple chunks without compromising crash safety.

V. RELATED WORK

Durability techniques in modern data stores fall into two broad families. **Write-ahead logging with periodic snapshots**, popularized by the ARIES recovery algorithm [1], offers strong crash consistency but incurs state-proportional latency spikes whenever a full checkpoint is flushed to disk. Production systems such as PostgreSQL and Cassandra still expose this behavior once their datasets reach tens of gigabytes.

Log-structured designs—originating with the Log-Structured File System [2] and later refined into Log-Structured Merge-trees (LSM) [?]¹—transform every update into a sequential append, greatly improving steady-state throughput. However, background compaction and segment cleaning re-introduce stalls that grow with data volume.

Work on **incremental or chunk-based checkpointing** tries to break this dependency by persisting only the delta since the last snapshot. Apache Flink’s incremental checkpoints [4] and Oracle’s buffer-pool increments [5] shorten pause times but still require occasional full materialization.

Remember Glaze extends the chunked idea one step further: the system never dumps the complete state. Instead, it commits exactly one fixed-size chunk per batch into a pre-allocated circular ring log. This choice guarantees constant-time commits, a deterministically bounded on-disk footprint, and native support for bulk, non-interdependent operations—properties that, to our knowledge, no prior scheme achieves simultaneously.

VI. CONCLUSION & FUTURE DEVELOPMENT

Future Work: The evaluation presented in this project demonstrates that *Remember Glaze* successfully meets its primary design goals, while also uncovering areas for further improvement and research. Here are a few ideas that could be explored:

a) *Memory Bandwidth as the Next Bottleneck:* Beyond certain chunk sizes that exceed the last-level cache, the bottleneck shifts from SSD write speeds to DRAM bandwidth and checksum computation performance. Testing on the AWS c7gd.8xlarge instance revealed that memory copying and checksum computations saturate around 180GB/s, tellingly outpacing the SSD’s maximum throughput of approximately 11GB/s. This indicates that improvements in storage speed alone will not boost overall throughput without first addressing memory bandwidth constraints, potentially through optimized data paths like non-temporal memory stores or SIMD instructions.

b) *Checksum Validation Overhead:* Checksum validation, specifically full-chunk CRC32 checks, introduces a substantial overhead (8–15%) at chunk sizes of 4–8 MB. Future optimizations, such as offloading checksum computation to background threads or exploiting ARMv9 hardware CRC instructions, could significantly reduce this overhead without compromising fault detection reliability. 4–8 MB.

c) *Enhancing Recovery Performance:* As shown in Experiment 3, recovery performance degrades more quickly compared to traditional WAL-snapshot approaches for very large states. A promising solution that we identified in this report is the concept of *rolling super-chunks*. Periodically saving a XOR-folded auxiliary snapshot of k contiguous chunks can reduce the required replay passes from N to N/k , incurring only $1/k$ additional write amplification cost.

Conclusion: This bachelor project aimed to reconcile three traditionally conflicting objectives within persistent storage systems: *high throughput*, *predictable latency*, and *robust crash-safety*. The developed solution, *Remember Glaze*, successfully integrates these goals using a chunk-oriented checkpoint ring log architecture. This design limits synchronous I/O operations to fixed-size regions, delivering predictable performance and enhanced durability.

d) *Main Findings:*

- Strict separation between in-memory computation and asynchronous durability ensures constant-time batch commits, unaffected by the overall state size.
- Testing on AWS c7gd.8xlarge demonstrated that *Remember Glaze* reduces batch latency by 40–70 % compared

to WAL-snapshot, eliminating snapshot-induced interruptions.

- Disk space utilization remains predictably bounded, effectively addressing the unlimited growth typical of traditional write-ahead logging systems.

e) *Limitations:* Recovery times scale significantly with an increasing number of chunks, and operations spanning multiple chunks must adhere to strict spatial-locality constraints.

f) *Summary:* *Remember Glaze* illustrates that careful selection of chunk granularity combined with a structured circular log can deliver robust performance and reliability with moderate implementation complexity. The principles and results presented here establish a solid basis for future developments in high-performance persistent data storage systems.

REFERENCES

- [1] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.
- [2] M. Rosenblum and J. K. Ousterhout. The design and implementation of a log-structured file system. *ACM Transactions on Computer Systems*, 10(1):26–52, 1992.
- [3] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil. The log-structured merge-tree (LSM-Tree). *Acta Informatica*, 33(4):351–385, June 1996.
- [4] P. Carbone, S. Ewen, G. Fóra, S. Haridi, and K. Tzoumas. Lightweight asynchronous snapshots for distributed dataflows. In *Proceedings of the 2015 Symposium on Cloud Computing (SoCC)*, pages 421–435. ACM, 2015.
- [5] Oracle Corporation. Checkpointing in Oracle 8.0. In *Proceedings of the 24th VLDB Conference*, pages 665–676. ACM, 1998.
- [6] Avinash Lakshman and Prashant Malik. Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review*, 44(2):35–40, 2010.